

Herramientas y requisitos de calidad de software

Semana 6





Contenido de la Semana 6

1. Herramientas de Calidad de Software

Análisis estático, pruebas funcionales, rendimiento y gestión de defectos

2. Requisitos de Calidad: ISO/IEC 25010

Las 8 características del modelo de calidad del producto

3. Caso de Estudio: Sistema Bancario de Créditos

Diagnóstico, intervención y resultados en un sistema real de mediana complejidad

Herramientas de Calidad: Analisis Estatico deCodigo

SonarQube

Analiza el codigo fuente sin ejecutarlo. Detecta duplicacion, deuda tecnica, vulnerabilidades y violaciones a reglas de codificacion.

Cuando usarlo: En cada pull request dentro del pipeline de integracion continua (CI). Su valor esta en la tendencia, no en el reporte puntual.

Kiuwan

Alternativa SaaS a SonarQube. Util cuando la infraestructura propia es limitada. Ofrece analisis en la nube y plugins para IDEs populares.

Limitacion clave: estas herramientas NO detectan errores de logica de negocio ni verifican que los requisitos se cumplan.

Herramientas de Calidad: Pruebas Funcionales y de Regresion

Selenium

Automatiza la interacción con interfaces web a nivel de navegador. Caso de uso central: pruebas de regresión para proteger comportamiento ya validado.

No es una herramienta exploratoria. Es para garantizar que lo que funcionaba siga funcionando tras cada nuevo despliegue.

Cypress

Competidor moderno de Selenium con mejor experiencia para aplicaciones JavaScript. La elección entre uno y otro depende del stack tecnológico, no de superioridad absoluta.

Principio fundamental: ninguna herramienta reemplaza criterios de aceptación bien definidos. Una herramienta mide; los criterios dicen que significa pasar o fallar.

Herramientas de Calidad: Rendimiento y Gestion de Defectos

Apache JMeter

Simula carga concurrente sobre servicios web y APIs. Responde preguntas concretas: cuantos usuarios simultaneos soporta el sistema antes de degradarse y donde esta el cuello de botella.

Cuando usarlo: antes de lanzamientos importantes o al detectar problemas de rendimiento en produccion. Sin preguntas definidas, JMeter produce datos que nadie sabe interpretar.

JIRA

Punto de integracion donde la calidad se vuelve visible para el negocio. Un defecto sin registrar en JIRA no existe para la gestion. Su valor esta en la trazabilidad: requisito -> caso de prueba -> defecto -> commit.



Requisitos de Calidad: ISO/IEC 25010

Que es ISO/IEC 25010?

Sucesor de ISO/IEC 9126. Define el modelo de calidad del producto de software en 8 características. No son una lista de deseos: cada una debe traducirse en metricas medibles para tener valor operativo.

Relacion con CMMI:

ISO/IEC 25010 define QUE atributos tiene un producto de calidad.

CMMI define COMO maduran los procesos que producen ese producto.

Son complementarios, no alternativos.

Las 8 características del modelo:

Adecuacion funcional | Fiabilidad | Usabilidad | Eficiencia de desempeno | Mantenibilidad | Seguridad | Compatibilidad | Portabilidad

ISO/IEC 25010: Características del Producto (1/2)

1. Adecuación Funcional

El sistema hace lo que debe hacer: completitud, corrección y pertinencia funcional. Un sistema puede ejecutar perfectamente una función que nadie necesita.

2. Fiabilidad

Capacidad de mantener el desempeño bajo condiciones definidas. Incluye disponibilidad, tolerancia a fallos y recuperación. Un 99.9% de disponibilidad = 8.7 horas de inactividad al año.

3. Usabilidad

No es diseño gráfico. Es la eficacia, eficiencia y satisfacción con que usuarios específicos logran objetivos específicos en un contexto específico.

4. Eficiencia de Desempeño

Comportamiento temporal, utilización de recursos y capacidad bajo condiciones establecidas. Aquí entra JMeter como herramienta de medición.



ISO/IEC 25010: Características del Producto (2/2)

5. Mantenibilidad

Modularidad, reusabilidad, analizabilidad y capacidad de modificación. SonarQube mide estos atributos a través de complejidad ciclomática y acoplamiento.

6. Seguridad

Confidencialidad, integridad, no repudio y autenticidad. La característica más subestimada en etapas tempranas y la más cara de corregir tardamente.

7. Compatibilidad

Coexistencia e interoperabilidad. Relevante en sistemas que se integran con legado o servicios de terceros.

8. Portabilidad

Adaptabilidad, instalabilidad y capacidad de reemplazo entre diferentes entornos de operación.

Como Redactar Requisitos de Calidad Medibles

Problema comun: requisitos vagos vs. requisitos medibles

Vago: "El sistema debe ser rapido."

Medible: "El tiempo de respuesta de la consulta de estado no debe superar 2 segundos con 200 usuarios concurrentes, medido con JMeter en entorno de staging."

Estructura de un requisito de calidad bien formado:

Caracteristica ISO 25010 + Metrica especifica + Herramienta de medicion + Umbral de aceptacion + Condiciones del entorno

Ejemplo - Mantenibilidad: La complejidad ciclomatica de ningun metodo nuevo debe superar 15, verificado automaticamente por SonarQube con quality gate que bloquea el merge si se viola esta regla.



Caso de Estudio: BancoRegional SA - Contexto

Sistema: Plataforma web de solicitud y seguimiento de créditos personales

18 meses en producción | 3.200 usuarios activos mensuales | Equipo de 7 desarrolladores

Problemas reportados por gerencia:

Tiempos de respuesta lentos en horas pico (Eficiencia de desempeño - ISO 25010)

Defectos que reaparecen tras cada corrección (Fiabilidad - ISO 25010)

Código difícil de entender para nuevos integrantes (Mantenibilidad - ISO 25010)

Objetivo del caso: mapear cada problema a una característica ISO 25010, aplicar la herramienta correcta y redefinir requisitos de calidad medibles.

Caso de Estudio: Diagnostico con Herramientas

SonarQube sobre el repositorio:

Deuda tecnica: 340 horas | Complejidad ciclomatica promedio: 28 (umbral recomendado: 10) | 23 bloques de codigo duplicado en controladores de solicitud.

Conclusion: la mantenibilidad es objetivamente baja. Explica por que los nuevos analistas no pueden leer el codigo.

JMeter - 150 usuarios concurrentes:

Con 20 usuarios: 1.2 seg | Con 150 usuarios: 8.9 seg | 12% de peticiones fallan por timeout.

Cuello de botella identificado: consulta SQL sin indice en tabla de movimientos (40.000 registros/dia).

Selenium - cobertura de pruebas:

Flujo de solicitud de credito: cubierto. Flujo de modificacion de solicitud ya enviada: sin cobertura. Ese flujo sin cubrir es exactamente donde reaparecen los defectos.

Caso de Estudio: Intervencion y Resultados

Eficiencia de desempeno - Solucion inmediata:

Indice anadido a tabla de movimientos (4 horas de trabajo). JMeter confirma: tiempo de respuesta con 200 usuarios baja de 8.9 a 1.4 seg. Timeouts: 0%.

Mantenibilidad - Sprint siguiente:

Modulos de scoring refactorizados. Complejidad ciclomatica promedio: 28 -> 11. Nuevos analistas reportan que pueden modificar el codigo de forma autonoma.

Fiabilidad - Cobertura Selenium ampliada:

Flujo de modificacion de solicitud cubierto. En el sprint siguiente, 3 defectos detectados en CI que antes llegaban a produccion son bloqueados automaticamente.

Leccion del caso: la evaluacion reactiva (encuesta de satisfaccion) no detecta estos problemas hasta que son visibles para el usuario. Las herramientas actuan antes.

Pruebas de Aceptacion: Concepto y Ubicacion en el Proceso

Que son las pruebas de aceptacion?

Verifican que el sistema cumple los criterios acordados con el cliente o usuario antes de ser liberado a produccion. No buscan bugs internos: validan comportamiento observable desde fuera del sistema.

Diferencia clave con otras pruebas:

- Unitarias: prueban una funcion o metodo aislado (desarrollador).
- Integracion: verifican que modulos se comunican correctamente (equipo tecnico).
- Aceptacion: verifican que el sistema hace lo que el negocio necesita (cliente + QA).

Tipos de pruebas de aceptacion:

- UAT (User Acceptance Testing): el usuario final valida flujos de negocio completos.
- Alpha/Beta: pruebas con usuarios reales en entorno controlado antes del lanzamiento.
- Contractual/Regulatoria: verifica cumplimiento de SLAs o normativas (ej. PCI-DSS en bancos).

Pruebas de Aceptacion: Criterios y Relacion con ISO/IEC 25010

Como se definen los criterios de aceptacion?

Cada criterio debe seguir el patron: DADO [contexto] CUANDO [accion] ENTONCES [resultado esperado]. Este es el lenguaje Gherkin, que conecta directamente con herramientas de automatizacion como Karate.

Ejemplo - sistema de creditos BancoRegional:

```
DADO que un asesor envia una solicitud de credito valida
CUANDO el sistema procesa la solicitud
ENTONCES la API retorna HTTP 201 con el ID de la solicitud en menos de 2 segundos
```

Mapecto a ISO/IEC 25010:

El criterio anterior valida Adecuacion Funcional (HTTP 201 correcto) + Eficiencia de Desempeno (2 seg). Un criterio bien escrito puede cubrir varias características simultaneamente.

Karate DSL: Pruebas de API en Backend Java

Que es Karate DSL?

Framework open source construido sobre Cucumber/JVM. Permite escribir pruebas de API REST/SOAP en sintaxis Gherkin sin necesidad de código Java adicional. Se integra nativamente con Maven/Gradle y genera reportes HTML detallados.

Por que Karate para un backend Java?

- No requiere cliente HTTP externo (Postman, curl): la llamada HTTP esta dentro del .feature.
- Validación de JSON/XML con expresiones concisas: `match response.status == "APROBADO"`.
- Soporte nativo para autenticación (Bearer token, Basic Auth, OAuth2).
- Ejecución paralela de escenarios sin configuración adicional.
- Se ejecuta en el mismo pipeline Maven que las pruebas unitarias (JUnit 5).

Comparado con REST Assured: Karate requiere menos código boilerplate y permite que QA no-desarrolladores escriban pruebas. REST Assured es preferible cuando se necesita lógica Java compleja dentro de la prueba.

Karate en Practica: Probando Rutas del Backend Java

Estructura de un archivo .feature con Karate:

Feature: Solicitud de credito - BancoRegional API

Background:

```
* url 'http://api.bancor.local/v1'
* header Authorization = 'Bearer ' + token
```

Scenario: Crear solicitud de credito valida

```
Given path '/creditos'
```

```
And request { monto: 5000000, plazo: 36, cedula: '12345678' }
```

```
When method POST
```

```
Then status 201
```

```
And match response.estado == 'EN_REVISION'
```

```
And match response.id == '#notnull'
```

Ejecucion con Maven: `mvn test -Dtest=CreditoRunner` | Reporte: <target/karate-reports/index.html>

Conclusiones

1. Las herramientas de calidad actúan en capas distintas del proceso: ninguna reemplaza a las demás ni a los criterios de aceptación bien definidos.
2. ISO/IEC 25010 define QUE es calidad en el producto; CMMI define COMO maduran los procesos que lo producen. Son complementarios.
3. Un requisito de calidad sin métrica, herramienta de medición y umbral de aceptación es una declaración de intención, no un requisito verificable.
4. La evaluación reactiva (encuesta post-despliegue) no detecta problemas de mantenibilidad ni cuellos de botella hasta que son visibles para el usuario. Las herramientas instrumentadas actúan antes.
5. El caso BancoRegional demuestra que problemas críticos de rendimiento, mantenibilidad y fiabilidad se resuelven con cambios precisos, no con reescrituras totales, cuando el diagnóstico es basado en datos.

Estándares y Métricas de Calidad en el Desarrollo de Software

Herramientas y requisitos de calidad de software

